

LLM API SELECTION AND PROMPT REFINEMENT

CO 2 AT-3

Name : Shaik Abdul Avez

Reg no:192472260

Course : CSA6503

Assignment Questions: 29 and 33

29. Choosing an LLM API

Introduction

A startup developing an AI chatbot must choose an LLM API that provides good model quality, reliable access, easy integration, suitable customization, manageable deployment, reasonable cost, and strong support for function calling and structured outputs. Three common choices are OpenAI, Google Gemini, and Hugging Face. Each has a different strength: OpenAI emphasizes a polished developer platform and agentic capabilities, Gemini provides strong integration with Google's AI ecosystem and competitive pricing, while Hugging Face provides broad model choice and greater flexibility around open and deployable models.

1. Model Availability

OpenAI provides a focused family of current models through its API. Its model catalog includes frontier, balanced, and cost-sensitive options, and current models support capabilities such as text and image input, multilingual use, vision, function calling, and structured outputs. This makes model selection relatively simple for a startup that wants a managed API.

Gemini provides multiple model families through the Gemini Developer API. Google offers models optimized for different requirements such as reasoning, speed, high-volume processing, multimodal input, and cost efficiency. The ecosystem also includes tools such as Google Search grounding, Maps, file search, and code execution for supported models.

Hugging Face is different because it is primarily an ecosystem and model hub rather than one single LLM family. It provides access to many open and third-party models through Inference Providers, and organizations can also deploy selected models on dedicated infrastructure. This gives the startup much more choice, but it also requires more evaluation to decide which model is best.

2. API Access and Integration

OpenAI offers SDKs and APIs designed specifically for application developers. This reduces the amount of infrastructure code required to connect a chatbot to a backend, database, or external tools. It is particularly suitable when the startup wants to move quickly from prototype to production.

Gemini provides the Gemini API and Google AI Studio, with SDK support and integration into Google's broader cloud and AI ecosystem. A startup already using Google Cloud services may benefit

from this ecosystem. Gemini also supports multimodal applications and a growing collection of built-in tools.

Hugging Face provides APIs and SDKs through Inference Providers and can route requests to different supported providers. This can simplify experimentation across models. For deeper control, Hugging Face Inference Endpoints allow organizations to deploy models on selected infrastructure, although this introduces additional deployment and operational considerations.

3. Customization

OpenAI is convenient when customization is mainly done through prompting, tool definitions, retrieval, and application-level orchestration. A startup can build a specialized chatbot without maintaining model infrastructure.

Gemini similarly supports customization through system instructions, tools, grounding, structured outputs, and application logic. Its integration with Google's ecosystem can be valuable for applications that depend on Google data or services.

Hugging Face has the strongest flexibility when the startup wants to experiment with different open models, fine-tune models, or control model deployment. This flexibility can be important for domain-specific applications, privacy-sensitive workloads, or organizations that want to reduce dependence on a single model provider.

4. Deployment

OpenAI and Gemini are primarily managed API approaches. The provider operates the underlying model infrastructure, so the startup does not need to manage GPUs or model servers. This reduces operational complexity and allows the team to concentrate on the chatbot application.

Hugging Face supports both hosted inference through providers and dedicated deployment through Inference Endpoints. Dedicated deployment gives greater control but requires the startup to consider infrastructure cost, scaling, monitoring, model updates, and availability.

5. Cost Considerations

Cost depends on the model selected, input and output token usage, context length, tool usage, caching, traffic volume, and deployment strategy. OpenAI provides multiple model tiers, including lower-cost models for high-volume applications. Gemini also provides free and paid tiers, and its current pricing includes lower-cost Flash and Flash-Lite options for high-volume workloads. Hugging Face can be

economical when using suitable open models, but dedicated endpoints introduce infrastructure charges based on the selected compute resources.

For a startup, token price alone should not determine the decision. The total cost of ownership should include engineering time, monitoring, infrastructure, model evaluation, API reliability, and the cost of changing providers later.

6. Function Calling and Structured Output

Function calling is important for chatbots that need to perform real actions, such as checking an order, creating a support ticket, booking an appointment, or retrieving account information. OpenAI provides strong support for function calling and structured outputs. Current OpenAI model comparisons explicitly list function calling and structured outputs as supported features.

Gemini also supports function calling and structured outputs, making it suitable for agentic applications that need predictable data or external tools. This is useful when a chatbot must connect to business APIs and databases.

Hugging Face supports structured outputs with compatible Inference Providers and models. Its function and tool capabilities depend more strongly on the selected model and provider, so compatibility must be checked before choosing a particular model. This is an important difference from a more standardized managed API.

7. Comparison Table

Criteria	OpenAI	Gemini	Hugging Face
Model availability	Focused current model family	Multiple Gemini model tiers	Very broad model ecosystem
API access	Mature managed API and SDKs	Gemini API, AI Studio and Google ecosystem	Inference Providers and SDKs
Customization	Prompting, tools, retrieval and application logic	Prompting, tools, grounding and ecosystem integration	High flexibility; open models and fine-tuning options
Deployment	Managed by provider	Managed by Google	Hosted providers or dedicated endpoints

Cost	Multiple model price tiers	Free and competitive paid tiers	Provider pricing or infrastructure-based endpoint cost
Function calling	Strong and standardized	Strong support	Depends on model/provider
Structured output	Strong support	Strong support	Supported for compatible models/providers
Best fit	Fast production chatbot	Google ecosystem and cost-sensitive apps	Maximum model flexibility and control

8. Recommendation

For a typical startup that wants to build and launch an AI chatbot quickly, OpenAI is the recommended option. The main reasons are its mature developer experience, clear model choices, strong function calling and structured-output support, managed deployment, and straightforward integration with application backends.

Gemini is a strong alternative, especially if the startup already relies heavily on Google Cloud or needs Google's search, Maps, or other ecosystem capabilities. Hugging Face is the best choice when model flexibility, open-model experimentation, fine-tuning, or deployment control is more important than minimizing engineering effort.

Therefore, OpenAI is the most suitable default choice for a startup focused on rapid development and reliable production integration. The final decision should still be validated using the startup's actual chatbot workload, latency requirements, quality tests, privacy requirements, and expected monthly token volume.

33. Prompt Refinement

Introduction

Prompt refinement is the process of improving an instruction given to an LLM so that the model produces a more accurate, relevant, consistent, and useful response. A weak prompt may be vague or incomplete, while a refined prompt provides the task, context, constraints, desired format, and relevant examples.

1. Importance of Prompt Refinement

LLMs generate responses based on the instructions and context provided to them. If the instruction is unclear, the model may produce a response that is too general, incorrectly formatted, or missing important information. Prompt refinement reduces ambiguity and gives the model a clearer target.

- Improves response accuracy and relevance.
- Reduces ambiguity and unwanted output.
- Controls the tone, length, and format.
- Makes results more consistent across repeated requests.
- Helps the model use supplied context correctly.
- Reduces unnecessary token usage and processing.

2. Elements of a Good Prompt

A refined prompt normally contains five important elements: role, task, context, constraints, and output format. The role tells the model what perspective it should use. The task describes exactly what needs to be done. Context supplies the information required to perform the task. Constraints define limitations such as length or allowed content. The output format specifies how the final response should be presented.

3. Example of Prompt Refinement

Weak prompt: 'Explain customer support.'

This prompt is too broad because it does not identify the audience, required depth, or expected format.

Refined prompt: 'Explain the purpose of a customer support ticket system for a college assignment. Describe how tickets are created, assigned, tracked, and resolved. Use simple technical language, organize the answer with headings, and keep it within 300 words.'

The refined version gives the model a clear task, context, audience, scope, style, and output limit. As a result, the response is more likely to match the user's actual requirement.

4. Techniques for Prompt Refinement

- Be specific: Replace vague instructions with measurable requirements.
- Provide context: Include the information the model needs to answer correctly.
- Define the output: Specify whether the answer should be a table, list, paragraph, JSON, or code.
- Use constraints: State word limits, required points, excluded content, or formatting rules.
- Give examples: Include a sample input and expected output when the task is complex.
- Break complex tasks into steps: Ask the model to perform related operations in an ordered way.
- Iterate and test: Compare responses and refine the prompt based on errors or missing information.

5. Prompt Refinement for Function Calling

When a chatbot uses function calling, the prompt should clearly explain when a function should be used and what information is required before calling it. For example, a customer support chatbot can be instructed to use `get_ticket_status` when the user asks for a ticket's status and to request the ticket ID if it is missing.

A good function-calling prompt should also prevent the model from inventing values. If a required parameter is unavailable, the chatbot should ask the user for that information instead of guessing. This improves reliability and reduces incorrect database operations.

6. Conclusion

Prompt refinement is an essential skill for developing reliable LLM applications. A refined prompt gives the model clear instructions, relevant context, constraints, and an expected output format. When combined with function calling and structured outputs, prompt refinement helps transform a general chatbot into a predictable application that can interact safely with business systems.

References

- OpenAI API documentation – Models, function calling and structured outputs.
- Google Gemini API documentation – Models, tools and pricing.

- Hugging Face documentation – Inference Providers, structured outputs and deployment.